# Actual HW3

Sam Olson

## Q1

Using the mixture model (1) and (2) fit the data in Data Set 1 and Data Set 2 separately using maximum likelihood. Produce approximate 95% confidence intervals for the modes of the distributions – these are the value of the parameter $\xi$ in (2). Conduct any additional analysis needed to construct a likelihood ratio test of a reduced model in which the parameters $\xi$ and $\theta$ are identical for both data sets against a full model in which those parameters differ between the two heights. Graph the extreme value density or densities for whichever model (full or reduced) you select.

Data model:

$$f(y \mid \mu_i) = \frac{1}{(2\pi)^{1/2}} \exp\left[-\frac{1}{2}(y-\mu_i)^2\right]. \tag{1}$$

Mixing distribution:

$$g(\mu \mid \xi, \theta) = \frac{1}{\theta} \exp\left(-\frac{\mu-\xi}{\theta}\right) \exp\left[-\exp\left(-\frac{\mu-\xi}{\theta}\right)\right], \qquad \theta > 0. \tag{2}$$

### A:

#### Direct Likelihood Evaluation

Table 1: Maximum likelihood estimates and Wald 95% confidence intervals for the mode parameter $\xi$.

| Height | $\hat{\xi}$ | Lower 95% | Upper 95% |
|---|---|---|---|
| 50m | 4.392 | 3.719 | 5.066 |
| 80m | 6.717 | 5.646 | 7.787 |

#### Profile Likelihood Evaluation

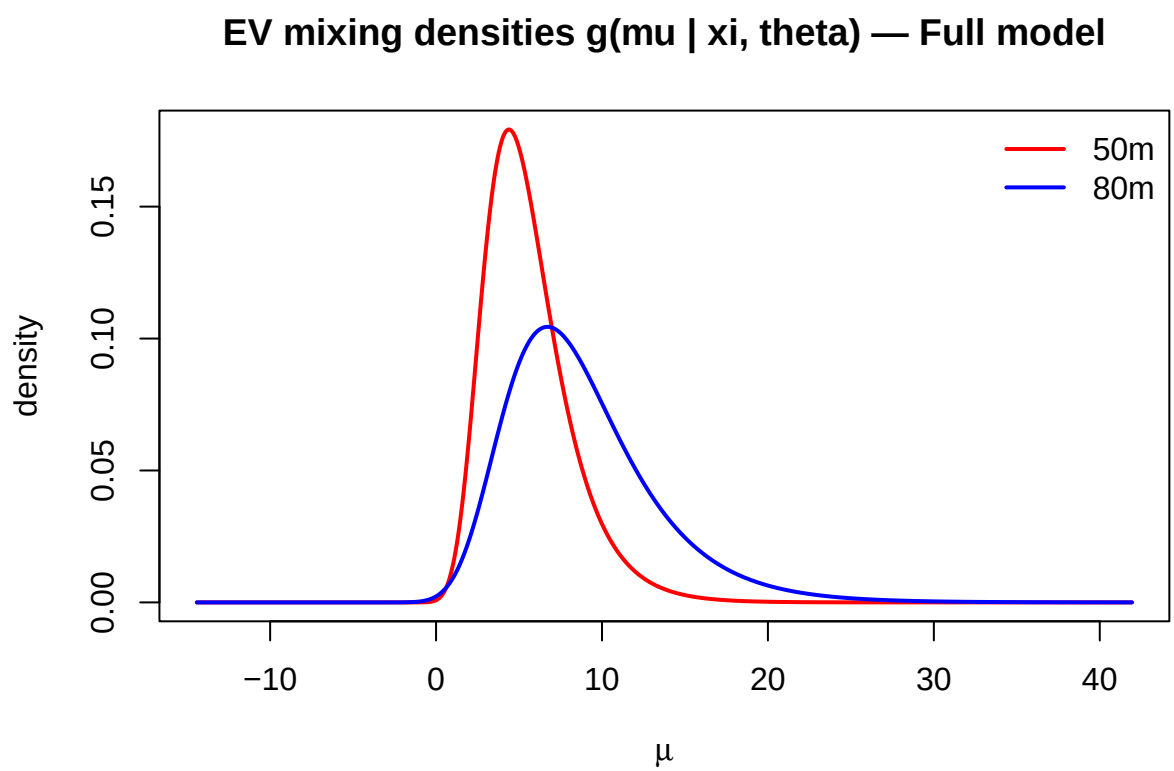
Table 2: Profile likelihood ratio 95% confidence intervals for the mode parameter $\xi$.

| Height | 95% Profile-LR CI for $\xi$ |
|---|---|
| 50m | [3.700, 5.075] |
| 80m | [5.633, 7.823] |

**Likelihood Ratio Test**

Table 3: Likelihood ratio test of equality of $(\xi, \theta)$ across heights (Full vs. Reduced model).

| Hypothesis | LR Statistic | df | $p$-value |
|---|---|---|---|
| $H_0$: common $(\xi, \theta)$ across heights | 18.579 | 2 | 9.24e-05 |



## Q2

In (2), $E(\mu) \approx \xi + 0.577\theta$, where 0.577 is Euler's constant (don't confuse with Euler's number). Using this expression, produce maximum likelihood estimate or estimates and 95% confidence interval or confidence intervals for the expected true maximum wind speed from your final model. Note that this will also be the expected observed maximum wind speed.

### A:

Table 4: Estimated expected true wind speed $E(\mu)$ and delta-method (Wald) 95% confidence intervals.

| Height | $\widehat{E(\mu)}$ | SE | Lower 95% | Upper 95% |
|---|---|---|---|---|
| 50m | 5.577 | 0.396 | 4.802 | 6.353 |
| 80m | 8.750 | 0.651 | 7.474 | 10.026 |

## Q3

Find a maximum likelihood estimate or estimates (depending on your final model) of the median daily maximum wind speed or speeds. Compute the associated 95% approximate confidence interval or intervals.

### A:

Table 5: Estimated median wind speed and delta-method (Wald) 95% confidence intervals.

| Height | $\widehat{\text{Median}}(Y)$ | SE | Lower 95% | Upper 95% |
|---|---|---|---|---|
| 50m | 5.208 | 0.367 | 4.488 | 5.928 |
| 80m | 8.049 | 0.601 | 6.871 | 9.227 |

# Q4

Find a maximum likelihood estimate or estimates (depending on your final model) for the probability that true daily maximum wind speed is between 4 and 17 m/s. Compute 95% approximate confidence intervals for the estimate or estimates.

## A

Table 6: Estimated probability $P(4 \leq \mu \leq 17)$ with 95% Wald confidence intervals.

| Height | $P(4 \widehat{\leq \mu \leq} 17)$ | SE | Lower 95% | Upper 95% |
|---|---|---|---|---|
| 50m | 0.700 | 0.060 | 0.583 | 0.817 |
| 80m | 0.832 | 0.046 | 0.743 | 0.922 |

## Q5

What would you conclude about the question of whether there is or is not a benefit to placing turbine hubs at either 50 or 80 m in height?

### A:

In Q1, the likelihood ratio test provided evidence that the wind speed distributions at 50m and 80m differ, leading us to select the full model with separate parameter estimates for each height. The estimated parameters imply systematically different wind speed behavior across heights.

In particular, the estimated probability that true daily maximum wind speed lies in the operationally relevant range of 4–17 m/s is larger at 80m than at 50m. The corresponding confidence intervals indicate that this difference is not attributable to sampling variability alone. Additionally, the estimated mean and median daily maximum wind speeds are higher at 80m.

Taken together, these results suggest that, under the fitted model and the conditions represented in the observed data, placing turbine hubs at 80m is associated with more favorable wind speed characteristics for turbine operation. This conclusion is conditional on the validity of the model assumptions and on future wind conditions being similar to those reflected in the data used for this analysis.

# Q6

As an exercise in learning about statistical procedures, your friend would like to see a Monte Carlo approximation to the expected true wind speed for the two heights separately. Produce such approximations along with 95% Monte Carlo intervals. Your friend understands intuitively that these approximations and the maximum likelihood estimates of Question 2 should have the same or nearly the same values, but marvels at the fact that the intervals associated with these approximations are so much more narrow than those computed using the asymptotic theory of likelihood estimation in Question 2. Explain why this is the case and what it means.

## A:

### Monte Carlo

Table 7: Monte Carlo approximation of $E(\mu)$ using $M$ simulated draws, with 95% Monte Carlo intervals.

| Height | $M$ | $\widehat{E(\mu)}_{\text{MC}}$ | Lower 95% | Upper 95% |
|---|---|---|---|---|
| 50m | 20000 | 5.583 | 5.546 | 5.620 |
| 80m | 20000 | 8.706 | 8.644 | 8.768 |

### Comparison - Concise

The Monte Carlo (MC) intervals are much narrower because they measure only simulation (numerical) error conditional on the estimated parameters $(\hat{\xi}, \hat{\theta})$, whereas the asymptotic confidence intervals in Question 2 account for statistical uncertainty in the joint estimation of $(\xi, \theta)$ from finite data.

### Comparison - More Details

For each height, we simulated

$$\mu^{(1)}, \ldots, \mu^{(M)} \sim \text{EV}(\hat{\xi}, \hat{\theta})$$

and estimated the expected true wind speed by

$$\widehat{E}_{MC}(\mu) = \bar{\mu}_M = \frac{1}{M} \sum_{m=1}^{M} \mu^{(m)}.$$

By the Central Limit Theorem for simulation,

$$\bar{\mu}_M \approx \mathcal{N}\left(E(\mu), \frac{\text{Var}(\mu)}{M}\right),$$

so an approximate 95% Monte Carlo interval is

$$\bar{\mu}_M \pm 1.96 \frac{s_M}{\sqrt{M}}.$$

The crucial point is that this interval treats $(\hat{\xi}, \hat{\theta})$ as fixed and quantifies only Monte Carlo approximation error, which decreases at rate $1/\sqrt{M}$ and can be made arbitrarily small by increasing $M$.

In contrast, the confidence interval in Question 2 is based on asymptotic likelihood theory for

$$E(\mu) = \xi + \gamma\theta, \qquad \gamma \approx 0.577,$$

and reflects uncertainty in estimating $(\xi, \theta)$ from $n = 50$ observations. That parameter uncertainty propagates into uncertainty about $E(\mu)$.

Thus, the Monte Carlo intervals are narrow because they measure numerical precision, not inferential uncertainty. The wider intervals from Question 2 are the appropriate measures of statistical uncertainty about the expected wind speed. To obtain a simulation-based interval comparable to Question 2, one would need to incorporate parameter uncertainty (e.g., via a parametric bootstrap), rather than conditioning on fixed MLEs.

# Appendix

```r
library(tidyverse)
library(numDeriv)

# External optimization helpers (Newton-Raphson, etc.)
source("C:/Users/samue/OneDrive/Desktop/Iowa_State_PS/STAT_6010/PS3/newtraph.txt")
source("C:/Users/samue/OneDrive/Desktop/Iowa_State_PS/STAT_6010/PS3/bisect.txt")

# Read data
m50 <- read.table("C:/Users/samue/OneDrive/Desktop/Iowa_State_PS/STAT_6010/PS3/winddat1.txt",
                  header = TRUE)
m80 <- read.table("C:/Users/samue/OneDrive/Desktop/Iowa_State_PS/STAT_6010/PS3/winddat2.txt",
                  header = TRUE)

y50 <- as.numeric(m50$y)
y80 <- as.numeric(m80$y)

# Euler-Mascheroni constant
gamma <- 0.5772156649015329

# ------------------------------------------------------------
# Extreme value (right-skew Gumbel) density for mu
# ------------------------------------------------------------
g_mu <- function(mu, xi, theta) {
  z <- (mu - xi) / theta
  (1 / theta) * exp(-z - exp(-z))
}

# ------------------------------------------------------------
# Stable marginal density m(y | xi, theta)
#   Uses reparameterization z = (mu - xi)/theta
#   m(y) = int phi(y | xi + theta z, 1) * exp(-z - exp(-z)) dz
# ------------------------------------------------------------
m_y_stable <- function(y, xi, theta, zL = -10, zU = 30) {
  integrand <- function(z) {
    log_int <- dnorm(y, mean = xi + theta * z, sd = 1, log = TRUE) - z - exp(-z)
    exp(log_int)
  }
  val <- integrate(integrand, lower = zL, upper = zU,
                   rel.tol = 1e-8, abs.tol = 0, subdivisions = 5000)$value
  pmax(val, .Machine$double.xmin)
}

# Vectorized log m(y)
log_m_vec_stable <- function(y, xi, theta, zL = -10, zU = 30) {
  vapply(y, function(yy) log(m_y_stable(yy, xi, theta, zL, zU)), numeric(1))
}

# ------------------------------------------------------------
# Log-likelihood for one dataset
# Parameterization: par = (xi, logtheta)
# ------------------------------------------------------------
```

```r
logLik_one <- function(par, y, zL = -10, zU = 30) {
  xi <- par[1]
  theta <- exp(par[2])
  sum(log_m_vec_stable(y, xi, theta, zL = zL, zU = zU))
}

# Negative log-likelihood (for optimization routines)
nll_one <- function(par, y, zL = -10, zU = 30) {
  -logLik_one(par, y = y, zL = zL, zU = zU)
}

# ------------------------------------------------------------
# Moment-based starting values
#   Var(Y) = (pi^2/6) theta^2 + 1
#   E(Y)   = xi + gamma * theta
# ------------------------------------------------------------
ev_start <- function(y) {
  v <- var(y)
  theta0 <- sqrt(pmax((v - 1) * 6 / pi^2, 1e-6))
  xi0 <- mean(y) - gamma * theta0
  c(xi0, log(theta0))
}

# ------------------------------------------------------------
# newtraph() interface: returns (logLik, gradient, Hessian)
# ------------------------------------------------------------
make_ders_one <- function(zL = -10, zU = 30) {
  function(par, dat) {
    f <- function(pp) logLik_one(pp, y = dat, zL = zL, zU = zU)
    ll <- f(par)
    g <- grad(f, par)
    H <- hessian(f, par)
    list(ll, g, H)
  }
}

# ------------------------------------------------------------
# Fit one dataset using Newton-Raphson
# Returns MLEs, Wald CI for xi, and covariance matrix
# ------------------------------------------------------------
fit_one_newtraph <- function(y, zL = -10, zU = 30) {
  start <- ev_start(y)
  ders <- make_ders_one(zL = zL, zU = zU)
  out <- newtraph(ders, dat = y, x0 = start,
                  quiet = TRUE, print_final = FALSE,
                  step_scale = 0.5, max_halving = 60)
  par_hat <- as.numeric(out[[1]])
  xi_hat <- par_hat[1]
  theta_hat <- exp(par_hat[2])
  fInf <- out[[3]]               # (-H)^(-1) at solution
  se_xi <- sqrt(fInf[1, 1])
  ci_xi_wald <- c(xi_hat - 1.96 * se_xi,
                  xi_hat + 1.96 * se_xi)
```

```r
  list(xi_hat = xi_hat,
       theta_hat = theta_hat,
       logLik = out[[2]],
       fInf = fInf,
       se_xi = se_xi,
       ci_xi_wald = ci_xi_wald)
}

fit50 <- fit_one_newtraph(y50, zL = -100, zU = 100)
fit80 <- fit_one_newtraph(y80, zL = -100, zU = 100)

# ------------------------------------------------------------
# Profile likelihood CI for xi
# Solves: 2(ell_hat - ell_prof(xi)) = chi^2_{1,0.95}
# ------------------------------------------------------------
profile_logLik_xi <- function(xi_fixed, y, logtheta_center,
                              zL = -10, zU = 30) {
  obj <- function(logtheta)
    nll_one(c(xi_fixed, logtheta), y = y, zL = zL, zU = zU)
  opt <- optimize(obj,
                  interval = c(logtheta_center - log(20),
                               logtheta_center + log(20)))
  list(logLik = -opt$objective)
}

ci_xi_profile <- function(y, xi_hat, theta_hat, logLik_hat,
                          zL = -10, zU = 30, alpha = 0.05) {
  cutoff <- qchisq(1 - alpha, df = 1)
  target <- function(xi) {
    ell_prof <- profile_logLik_xi(
      xi_fixed = xi,
      y = y,
      logtheta_center = log(theta_hat),
      zL = zL, zU = zU
    )$logLik
    2 * (logLik_hat - ell_prof) - cutoff
  }
  step0 <- max(0.25, 0.5 * theta_hat)
  lower <- uniroot(target,
                   interval = c(xi_hat - 5 * step0, xi_hat))$root
  upper <- uniroot(target,
                   interval = c(xi_hat, xi_hat + 5 * step0))$root
  c(lower, upper)
}

ci50_prof <- ci_xi_profile(y50, fit50$xi_hat,
                           fit50$theta_hat, fit50$logLik,
                           zL = -100, zU = 100)

ci80_prof <- ci_xi_profile(y80, fit80$xi_hat,
                           fit80$theta_hat, fit80$logLik,
                           zL = -100, zU = 100)
```

```r
# ------------------------------------------------------------
# Likelihood ratio test: full vs reduced model
# Full: separate (xi, theta) by height
# Reduced: common (xi, theta)
# ------------------------------------------------------------
logLik_full <- function(par, y50, y80,
                        zL = -100, zU = 100) {
  xi1 <- par[1]; th1 <- exp(par[2])
  xi2 <- par[3]; th2 <- exp(par[4])
  sum(log_m_vec_stable(y50, xi1, th1, zL, zU)) +
    sum(log_m_vec_stable(y80, xi2, th2, zL, zU))
}

logLik_reduced <- function(par, y50, y80,
                           zL = -100, zU = 100) {
  xi <- par[1]; th <- exp(par[2])
  sum(log_m_vec_stable(y50, xi, th, zL, zU)) +
    sum(log_m_vec_stable(y80, xi, th, zL, zU))
}

full_opt <- optim(c(fit50$xi_hat, log(fit50$theta_hat),
                    fit80$xi_hat, log(fit80$theta_hat)),
                  fn = function(p) -logLik_full(p, y50, y80),
                  method = "BFGS")

red_opt <- optim(ev_start(c(y50, y80)),
                 fn = function(p) -logLik_reduced(p, y50, y80),
                 method = "BFGS")

LR <- 2 * (-full_opt$value - (-red_opt$value))
pval <- 1 - pchisq(LR, df = 2)
use_full <- (pval < 0.05)

# Plot EV mixing densities for selected model
if (use_full) {
  xi50_hat <- full_opt$par[1]
  th50_hat <- exp(full_opt$par[2])
  xi80_hat <- full_opt$par[3]
  th80_hat <- exp(full_opt$par[4])
  mu_grid <- seq(min(xi50_hat - 6 * th50_hat,
                     xi80_hat - 6 * th80_hat),
                 max(xi50_hat + 10 * th50_hat,
                     xi80_hat + 10 * th80_hat),
                 length.out = 800)
  plot(mu_grid, g_mu(mu_grid, xi50_hat, th50_hat),
       type = "l", col = "red", lwd = 2,
       xlab = expression(mu), ylab = "density")
  lines(mu_grid, g_mu(mu_grid, xi80_hat, th80_hat),
        col = "blue", lwd = 2)
} else {
  xi_hat <- red_opt$par[1]
  th_hat <- exp(red_opt$par[2])
  mu_grid <- seq(xi_hat - 6 * th_hat,
```

```r
                 xi_hat + 10 * th_hat,
                 length.out = 800)
  plot(mu_grid, g_mu(mu_grid, xi_hat, th_hat),
       type = "l", lwd = 2,
       xlab = expression(mu), ylab = "density")
}

# ------------------------------------------------------------
# Q2: E(mu) = xi + gamma * theta
# Delta-method CI
# ------------------------------------------------------------
wald_ci_mean_mu <- function(fit) {
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat
  V <- fit$fInf
  eta_hat <- xi_hat + gamma * th_hat
  grad_vec <- c(1, gamma * th_hat)
  se <- sqrt(drop(t(grad_vec) %*% V %*% grad_vec))
  ci <- eta_hat + c(-1, 1) * qnorm(0.975) * se
  list(eta_hat = eta_hat, se = se, ci95 = ci)
}

ci50_eta_wald <- wald_ci_mean_mu(fit50)
ci80_eta_wald <- wald_ci_mean_mu(fit80)

# ------------------------------------------------------------
# Q3: Median of observed Y under mixture
# Solve F_Y(y) = 0.5
# ------------------------------------------------------------
FY_stable <- function(y, xi, theta,
                      zL = -100, zU = 100) {
  integrand <- function(z)
    pnorm(y - (xi + theta * z)) *
    exp(-z - exp(-z))
  val <- integrate(integrand, lower = zL, upper = zU)$value
  pmin(pmax(val, 0), 1)
}

median_y <- function(xi, theta) {
  f <- function(y) FY_stable(y, xi, theta) - 0.5
  uniroot(f,
          interval = c(xi - 10 * theta,
                       xi + 10 * theta))$root
}

wald_ci_median_y <- function(fit) {
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat
  V <- fit$fInf
  h <- function(par) {
    xi <- par[1]
    th <- exp(par[2])
    median_y(xi, th)
```

```r
  }
  par_hat <- c(xi_hat, log(th_hat))
  m_hat <- h(par_hat)
  g <- grad(h, par_hat)
  se <- sqrt(drop(t(g) %*% V %*% g))
  ci <- m_hat + c(-1, 1) * qnorm(0.975) * se
  list(median_hat = m_hat, se = se, ci95 = ci)
}

ci50_med <- wald_ci_median_y(fit50)
ci80_med <- wald_ci_median_y(fit80)

# ------------------------------------------------------------
# Q4: P(4 <= mu <= 17) under EV mixing distribution
# ------------------------------------------------------------
G_mu <- function(u, xi, theta)
  exp(-exp(-(u - xi) / theta))

wald_ci_p_between_mu <- function(fit, a = 4, b = 17) {
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat
  V <- fit$fInf
  h <- function(par) {
    xi <- par[1]
    th <- exp(par[2])
    G_mu(b, xi, th) - G_mu(a, xi, th)
  }
  par_hat <- c(xi_hat, log(th_hat))
  p_hat <- h(par_hat)
  g <- grad(h, par_hat)
  se <- sqrt(drop(t(g) %*% V %*% g))
  ci <- p_hat + c(-1, 1) * qnorm(0.975) * se
  list(p_hat = p_hat, se = se, ci95 = ci)
}

ci50_p <- wald_ci_p_between_mu(fit50)
ci80_p <- wald_ci_p_between_mu(fit80)

# ------------------------------------------------------------
# Q6: Monte Carlo approximation of E(mu)
# ------------------------------------------------------------
set.seed(43)

rEV <- function(n, xi, theta) {
  u <- runif(n)
  xi - theta * log(-log(u))
}

mc_E_mu <- function(fit, M = 20000) {
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat
  mu_draws <- rEV(M, xi_hat, th_hat)
  mc_mean <- mean(mu_draws)
```

```r
  mc_se <- sd(mu_draws) / sqrt(M)
  mc_ci <- mc_mean + c(-1, 1) * qnorm(0.975) * mc_se
  plug_in <- xi_hat + gamma * th_hat
  list(mc_mean = mc_mean,
       mc_ci95 = mc_ci,
       plug_in = plug_in)
}

mc50 <- mc_E_mu(fit50)
mc80 <- mc_E_mu(fit80)
```